

6장. 합성곱 신경망 II

6.1 이미지 분류를 위한 신경망

이미지 분류(Image Classification): 입력 이미지에 특정 대상이 존재하는지 판단하거나, 이미지가 어떤 범주에 해당하는지 구분하는 작업

6.1.1 LeNet-5

1. LeNet-5의 개념과 구조

- 손글씨 숫자 인식을 위해 개발된 초기 합성곱 신경망으로, 이후 CNN 구조의 기반
- 합성곱과 다운 샘플링을 반복한 뒤 완전연결층에서 분류하는 구조
- 다운 샘플링(Sub-sampling): 특성 맵의 공간 크기를 줄이는 과정으로, 교재 예제에서는 최대 풀링 사용.

합성곱층의 특징 추출 → 풀링층의 크기 축소 → 완전연결층의 분류로 이어지는 기본 흐름.

2. LeNet-5 예시

32×32 이미지에 합성곱층과 최대 풀링층을 두 차례 적용한 뒤, 완전연결층을 이용하여 개와 고양이를 분류하는 예시

계층	특성 맵 수	출력 크기	커널 크기
입력 이미지	1	32×32	—
첫 번째 합성곱층	6	28×28	5×5
첫 번째 최대 풀링층	6	14×14	2×2
두 번째 합성곱층	16	10×10	5×5
두 번째 최대 풀링층	16	5×5	2×2
첫 번째 완전연결층	—	120	—
두 번째 완전연결층	—	84	—
출력층	—	2	—

- 합성곱층과 중간 완전연결층의 활성화 함수: ReLU 사용
- 최종 출력: 개와 고양이 두 종류를 구분하기 위한 출력값 2개

3. 이미지 전처리

전처리: 이미지를 모델이 입력받을 수 있는 형태로 변환하는 과정

구분	주요 처리
훈련 데이터	무작위 자르기, 좌우 반전, 텐서 변환, 정규화
검증 데이터	크기 조절과 중앙 자르기, 텐서 변환, 정규화

- **훈련 이미지의 무작위 변형:** 같은 이미지에서도 다양한 부분과 방향을 학습하도록 하는 처리
- **텐서 변환:** 이미지를 파이토치에서 연산할 수 있는 숫자 데이터로 변환
- **정규화:** 지정한 평균과 표준편차를 이용한 입력값 조정.

4. 데이터셋과 데이터로더

- **데이터셋(Dataset):** 이미지의 위치와 레이블을 관리하고, 요청받은 데이터를 읽어오는 역할
`__init__()` 에서 이미지 경로 저장, `__getitem__()` 에서 해당 이미지와 레이블 반환
- **데이터로더(DataLoader):** 데이터셋에서 배치 크기만큼 데이터를 가져와 모델에 전달하는 역할
전체 이미지를 한꺼번에 읽는 대신, 일정한 개수로 나누어 처리하는 방식

데이터셋에서 데이터를 읽는 방법 정의 → 데이터로더에서 배치 단위로 전달하는 흐름

5. 실습 예제

- 실습에서는 3×224×224의 RGB 이미지를 입력으로 사용

단계	데이터 형태
입력	3×224×224
첫 번째 합성곱	16×220×220
첫 번째 최대 풀링	16×110×110
두 번째 합성곱	32×106×106
두 번째 최대 풀링	32×53×53

형태 표기의 순서: 채널 수 × 높이 × 너비

마지막 특성 맵을 완전연결층에 전달하기 위한 입력값 개수는 **32×53×53 = 89,888개**.

Conv2d 계층에서 출력 크기 구하는 공식

$$\text{출력 크기} = (W - F + 2P) / S + 1$$

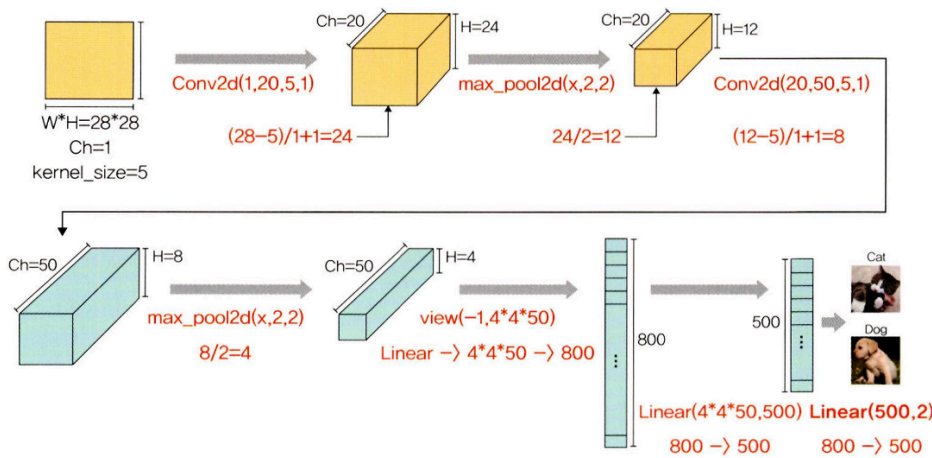
- W : 입력 데이터의 크기(input_volume_size)
- F : 커널 크기(kernel_size)
- P : 패딩 크기(padding_size)
- S : 스트라이드(strides)

MaxPool2d 계층에서의 출력 크기 구하는 공식

$$\text{출력 크기} = IF / F$$

- IF : 입력 필터의 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 합니다)
- F : 커널 크기(kernel_size)

각각의 계층에 대한 결과



6. 모델 구조와 학습 파라미터 확인

- `torchsummary.summary()`: 계층별 출력 형태와 파라미터 수 확인
- `model.parameters()`: 모델의 가중치와 바이어스 등 파라미터 접근
- `requires_grad=True` 인 파라미터의 개수 계산: 기울기 계산 대상으로 설정된 파라미터 수 확인

7. 손실 함수와 옵티마이저

항목	교재의 설정	역할
손실 함수	<code>CrossEntropyLoss()</code>	예측과 정답의 차이 계산
옵티마이저	SGD	계산한 기울기를 이용한 파라미터 갱신
학습률	0.001	파라미터 변경 크기 조절
모멘텀	0.9	이전 갱신 방향을 반영하여 진동 완화

모멘텀 SGD: 현재 기울기뿐 아니라 이전 이동 방향도 반영하여 파라미터를 갱신하는 방법.

8. 학습과 검증 과정

한 에포크 안에서 훈련과 검증을 각각 수행하고, 손실과 정확도를 기록하는 구성.

순서	주요 코드	역할
①	<code>model.train()</code>	훈련 모드 설정
②	<code>optimizer.zero_grad()</code>	이전 기울기 초기화
③	<code>model(inputs)</code>	입력에 대한 예측 계산
④	<code>criterion(outputs, labels)</code>	예측과 정답의 손실 계산
⑤	<code>loss.backward()</code>	파라미터별 기울기 계산
⑥	<code>optimizer.step()</code>	파라미터 갱신

- **검증 단계:** `model.eval()` 로 평가 모드 설정, 역전파와 파라미터 갱신 없이 성능 측정
- **정확도 계산:** 예측 클래스와 실제 레이블이 일치하는 데이터 수를 전체 데이터 수로 나누는 방식. 24

에포크 손실 계산의 핵심

`CrossEntropyLoss()` 의 기본 반환값은 **배치 내부의 평균 손실**. 따라서 각 배치의 평균 손실에 배치 데이터 수를 곱해 누적한 뒤, 전체 데이터 수로 나누어 에포크 평균 손실 계산.

9. 테스트와 예측 결과 저장

- `torch.no_grad()` 사용: 테스트 중 불필요한 기울기 계산 생략
- `unsqueeze(0)` 사용: 이미지 한 장에 배치 차원 추가 `[3, 224, 224]` → `[1, 3, 224, 224]` 의 형태 변환
- **소프트맥스 적용:** 모델의 출력 점수를 클래스별 확률로 변환하고, 이미지 ID와 예측값을 데이터 프레임 및 CSV로 저장.

6.1.2 AlexNet

1. AlexNet의 개념

- **ILSVRC 2012 이미지 인식 대회에서 우승한 CNN 구조**
- **합성곱층 5개와 완전연결층 3개로** 구성하며, 원래 모델에서는 1,000개 범주 분류
- **GPU 두 개를 이용한 병렬 처리**를 적용한 구조로, 합성곱과 풀링 이후 완전연결층에서 분류하는 흐름은 LeNet-5와 유사.

2. CNN 구조에서 깊이의 의미

CNN의 중간 데이터를 너비, 높이, 깊이로 표현

- **너비와 높이:** 이미지 또는 특성 맵의 공간 크기
- **깊이:** 채널 수에 해당하며, RGB 입력 이미지에서는 3
- **합성곱 이후의 깊이:** 생성된 특성 맵의 개수에 따라 변화

* 이때 특성 맵의 깊이인 채널 수와 신경망의 깊이인 계층 수는 서로 다른 의미

3. 구조 설명의 주요 구성

합성곱층	특성 맵 수	커널 크기
첫 번째	96	11×11
두 번째	256	5×5
세 번째	384	3×3
네 번째	384	3×3
다섯 번째	256	3×3

- **합성곱층의 활성화 함수:** ReLU 사용
- **일부 합성곱층 뒤의 최대 풀링:** 3×3 커널과 스트라이드 2 적용
- **완전연결층의 구성:** 4,096 → 4,096 → 1,000

첫 번째 합성곱에서는 **11×11×3 커널과 스트라이드 4**를 사용하여, **55×55 크기의 특성 맵 96개** 생성.

4. 실습코드 예제

개와 고양이 분류에 맞게 채널 수와 완전연결층을 조정한 AlexNet 구현 실습

구분	코드에서의 구성
<code>features</code>	합성곱, ReLU, 최대 풀링을 이용한 특징 추출
<code>avgpool</code>	특성 맵의 공간 크기를 6×6으로 조정
<code>classifier</code>	4,096 → 512 → 2의 완전연결층 구성

순전파 흐름

- 특징 추출 → 평균 풀링 → 특성 맵 펼치기 → 분류 점수 계산. 분류 부분에는 드롭아웃도 포함

5. 실습에서 설명하는 주요 기능

- `inplace=True`: 연산 결과를 별도 데이터로 만드는 대신, 기존 데이터를 결과값으로 변경하는 설정
- `AvgPool2d`: 커널 크기와 스트라이드 등을 지정하여 평균 풀링 수행
- `AdaptiveAvgPool2d`: 원하는 출력 크기를 지정하여 평균 풀링 수행

`AdaptiveAvgPool2d((6, 6))` 은 출력의 높이와 너비를 **6×6으로 맞추는 설정**. 입력 크기가 달라도 뒤쪽 완전연결층에 전달할 공간 크기를 맞추는데 유용한 기능.

6. 학습 과정

데이터 준비와 학습 함수의 전반적인 흐름은 LeNet 실습과 동일

- 입력 이미지 크기**: 실습 코드에서 256×256 사용
- 학습 설정**: 모멘텀 SGD와 교차 엔트로피 손실 사용
- 훈련 횟수**: 실행 환경과 학습 시간을 고려한 10에포크 설정

6.1.3 VGGNet

1. VGGNet의 개념과 설계 방향

- 신경망을 깊게 만드는 것이 성능에 미치는 영향을 확인하기 위한 모델
- 합성곱 필터 크기를 3×3으로 고정하여, 계층 깊이의 영향에 집중
- 작은 필터를 이용하여 합성곱층의 파라미터 수와 훈련 부담을 줄이려는 설계 방향

2. 반복적인 네트워크 구조

3×3 합성곱과 ReLU를 반복하고, 최대 풀링으로 공간 크기를 줄이는 구성

- 합성곱층**: 작은 커널을 이용한 특징 추출
- 최대 풀링층**: 2×2 커널을 통한 높이와 너비 축소
- 채널 구성**: 뒤쪽 블록으로 갈수록 64 → 128 → 256 → 512로 증가

3. 설정 리스트를 이용한 계층 정의

설정 리스트에 따라 합성곱층과 최대 풀링층을 자동으로 구성하는 방식

- 숫자**: 합성곱층의 출력 채널 수
- 'M'**: 최대 풀링층 적용
- 이전 층의 출력 채널**: 다음 합성곱층의 입력 채널로 사용

예를 들어 `[64, 'M', 128, 'M']` 은 다음 구성의 의미.

→ 64채널 합성곱 → 최대 풀링 → 128채널 합성곱 → 최대 풀링

VGG11, VGG13, VGG16, VGG19는 설정 리스트의 합성곱층 개수를 달리하여 구성. 직접 새로운 설정 리스트를 작성하면 사용자 정의 구조도 생성 가능한 방식.

4. 계층 생성 함수의 동작

`get_vgg_layers()` 에서 설정 리스트를 순회하며 계층 생성.

- **깊이 'M' 인 경우:** `MaxPool2d(kernel_size=2)` 추가
- **깊이 정수인 경우:** 해당 깊이를 출력 채널 수로 사용하는 `Conv2d` 추가
- **합성곱의 설정:** 커널 크기 3, 패딩 1 적용

합성곱 뒤에는 ReLU를 추가하고, 설정에 따라 배치 정규화 계층도 포함함

배치 정규화(Batch Normalization): 배치의 통계값을 이용하여 중간 데이터의 분포를 정규화하는 방법.

5. VGG 클래스의 전체 구성

구성 요소	역할
<code>features</code>	설정 리스트로 생성한 합성곱과 풀링 계층
<code>avgpool</code>	특성 맵의 공간 크기를 7×7로 조정
<code>classifier</code>	완전연결층을 통한 클래스별 점수 계산

분류 부분의 구성: 512×7×7개의 입력값 → 4,096 → 4,096 → 출력 클래스 수

중간 완전연결층 뒤에 ReLU와 드롭아웃 적용.

- **x 반환:** 최종 클래스별 출력 점수
- **h 반환:** 완전연결층에 전달하기 위해 펼친 특징 벡터

따라서 교재의 VGG 모델은 `return x, h` 로 두 값을 반환하며, 학습 코드에서는 `y_pred, _ = model(x)` 로 예측값만 사용하는 방식.

6. 사전 훈련된 모델 사용

사전 훈련된 모델: 다른 데이터로 미리 학습한 파라미터를 제공하는 모델

교재에서는 직접 구현한 VGG 구조와 함께, 이미 학습된 모델을 불러오는 방법도 소개.

- `vgg11_bn`: 배치 정규화가 포함된 VGG11 모델
- `pretrained=True`: 미리 학습된 파라미터를 불러오는 교재의 설정
- **직접 구현의 학습 목적:** 계층 구성 방법을 이해하고, 새로운 설정 리스트로 구조를 변경하는 능력 확보

7. 데이터 준비와 깊은 복사

ImageFolder 사용: 클래스별 하위 폴더로 구성된 이미지 데이터셋을 불러오는 방법

예를 들어 훈련 폴더 아래에 `Cat`, `Dog` 폴더가 있는 구조에서 각 폴더의 이미지를 불러오는 방식

- **훈련 데이터 전처리:** 크기 조절, 무작위 회전 등 적용
- **훈련·검증 분리:** `random_split()` 을 이용하여 데이터셋 단계에서 분리
- **검증 데이터 전처리 변경:** `copy.deepcopy()` 후 검증용 전처리 적용

복사 방식의 차이

방식	특징
단순 대입	두 변수가 같은 객체를 참조
얕은 복사	바깥 객체 복사, 내부 객체의 참조 공유
깊은 복사	내부 객체까지 별도로 복사

검증 데이터에 깊은 복사를 적용하는 이유는 **검증용 전처리를 변경할 때 훈련 데이터셋의 전처리까지 함께 바뀌는 상황 방지.**

데이터로더에서는 배치 크기 128을 사용하고, 훈련 데이터에만 `shuffle=True` 를 적용함

8. 학습과 평가

구분	주요 내용
옵티마이저	Adam 사용
손실 함수	교차 엔트로피 손실 사용
훈련 함수	예측과 손실 계산 후 역전파 및 파라미터 갱신
평가 함수	평가 모드에서 기울기 계산 없이 손실과 정확도 측정

- **예측 클래스 선택:** `argmax()` 로 클래스별 점수가 가장 큰 위치 선택

- **정확도 계산:** 예측과 실제 레이블이 일치하는 개수를 전체 개수로 나누는 방식
- **훈련과 평가의 차이:** 훈련에서는 `backward()` 와 `step()` 수행, 평가에서는 `model.eval()` 과 `torch.no_grad()` 사용

9. 모델 저장과 테스트

5에포크 동안 학습하며, 검증 손실이 가장 낮은 모델의 파라미터 저장

- `model.state_dict()` : 모델의 파라미터 상태 확인
- `torch.save()` : 검증 손실이 개선된 시점의 파라미터 저장
- `load_state_dict()` : 저장한 파라미터를 모델에 다시 적용한 뒤 테스트 수행

최종 에포크의 모델을 그대로 사용하는 것이 아니라, 검증 손실이 가장 낮았던 상태를 불러와 테스트하는 흐름

10. 예측 결과 확인과 시각화

테스트 이미지의 예측 확률을 계산하고, 실제 레이블과 비교하여 결과 확인

주요 기능	역할
<code>softmax()</code>	출력 점수를 클래스별 확률로 변환
<code>argmax()</code>	가장 높은 확률에 해당하는 클래스 선택
<code>torch.cat()</code>	배치별 이미지, 레이블, 예측 확률을 하나로 연결
<code>sort()</code>	예측 결과를 지정한 기준으로 정렬
<code>permute()</code>	이미지 출력에 필요한 축 순서 변경

이미지 시각화에서는 채널 × 높이 × 너비 형태를 높이 × 너비 × 채널 순서로 바꾸는 처리 필요. 이미지값의 범위 조정과 축 변경 후, 정확하게 예측한 이미지 출력.

밑줄이 붙은 연산의 의미

- `add_()` 처럼 메서드 끝에 `_` 가 있는 연산은 기존 텐서 자체를 변경하는 방식. 교재에서는 이미지값 조정 과정에서 이러한 연산 사용

6.1.4 GoogLeNet

1. GoogLeNet의 개념과 목적

- 하드웨어 자원을 효율적으로 이용하면서 학습 능력을 높이기 위한 깊고 넓은 신경망
- 인셉션 모듈(Inception Module)을 이용하여 여러 종류의 합성곱과 풀링 연산 수행

깊은 구조는 계층 수가 많은 구성, 넓은 구조는 한 계층의 뉴런 수가 많은 구성

2. 깊고 넓은 신경망의 문제

신경망을 깊고 넓게 만들 때 발생할 수 있는 문제.

- **과적합:** 훈련 데이터에 지나치게 맞춰져 새로운 데이터에서 성능이 떨어지는 문제
- **기울기 소멸:** 앞쪽 계층으로 전달되는 기울기가 작아져 학습이 어려워지는 문제
- **학습과 연산 부담:** 학습 시간 증가와 연산 속도 저하

⇒ GoogLeNet은 효율적으로 특징을 추출해 이러한 문제를 해결할 수 있음

3. 인셉션 모듈의 구성

- 하나의 모듈 안에서 서로 다른 합성곱과 풀링을 수행하는 네 가지 경로

경로	연산 구성
①	1×1 합성곱
②	1×1 합성곱 + 3×3 합성곱
③	1×1 합성곱 + 5×5 합성곱
④	3×3 최대 풀링 + 1×1 합성곱

- **다양한 커널 크기 사용:** 1×1, 3×3, 5×5 합성곱을 이용한 특징 추출
- **최대 풀링 경로 포함:** 합성곱 외에 풀링을 활용한 처리

- **풀링에 패딩 추가:** 모듈 내부에서 입력과 출력의 높이 및 너비를 동일하게 유지하기 위한 설정

4. 희소 연결의 아이디어

희소 연결(Sparse Connectivity): 모든 노드를 뻘뻘하게 연결하는 대신, 관련성이 높은 노드끼리 연결하는 방법

- **밀집 연결:** 많은 노드가 촘촘하게 연결된 구성
- **희소 연결:** 필요한 관련성을 중심으로 연결하는 구성
- **설계 목적:** 연산량을 줄이고 과적합을 완화하면서 특징을 효율적으로 추출하는 방향